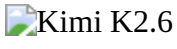
kimi-k2-6-hf-model-card

---

tags:

- compressed-tensors license: other license_name: modified-mit library_name: transformers pipeline_tag: image-text-to-text

---



Chat Kimi K2.6 Homepage Moonshot AI
Hugging Face Moonshot AI Twitter Kimi.ai Discord Kimi.ai ModelScope Moonshot AI
License Modified MIT

🤗 huggingchat | 📰 Tech Blog

1. Model Introduction

Kimi K2.6 is an open-source, native multimodal agentic model that advances practical capabilities in long-horizon coding, coding-driven design, proactive autonomous execution, and swarm-based task orchestration.

Key Features

- **Long-Horizon Coding**: K2.6 achieves significant improvements on complex, end-to-end coding tasks, generalizing robustly across programming languages (Rust, Go, Python) and domains spanning front-end, DevOps, and performance optimization.
- **Coding-Driven Design**: K2.6 is capable of transforming simple prompts and visual inputs into production-ready interfaces and lightweight full-stack workflows, generating structured layouts, interactive elements, and rich animations with deliberate aesthetic precision.
- **Elevated Agent Swarm**: Scaling horizontally to 300 sub-agents executing 4,000 coordinated steps, K2.6 can dynamically decompose tasks into parallel, domain-specialized subtasks, delivering end-to-end outputs from documents to websites to spreadsheets in a single autonomous run.
- **Proactive & Open Orchestration**: For autonomous tasks, K2.6 demonstrates strong performance in powering persistent, 24/7 background agents that proactively manage schedules, execute code, and orchestrate cross-platform operations without human oversight.

2. Model Summary

Architecture	Mixture-of-Experts (MoE)
Total Parameters	1T
Activated Parameters	32B
Number of Layers (Dense layer included)	61
Number of Dense Layers	1
Attention Hidden Dimension	7168
MoE Hidden Dimension (per Expert)	2048
Number of Attention Heads	64
Number of Experts	384

Selected Experts per Token	8
Number of Shared Experts	1
Vocabulary Size	160K
Context Length	256K
Attention Mechanism	MLA
Activation Function	SwiGLU
Vision Encoder	MoonViT
Parameters of Vision Encoder	400M

3. Evaluation Results

Benchmark	Kimi K2.6	GPT-5.4 (xhigh)	Claude Opus 4.6 (max effort)	Gemini 3.1 Pro (thinking high)	Kimi K2.5
Agentic					
HLE-Full (w/ tools)	54.0	52.1	53.0	51.4	50.2
BrowseComp	83.2	82.7	83.7	85.9	74.9
BrowseComp (Agent Swarm)	86.3				78.4
DeepSearchQA (f1-score)	92.5	78.6	91.3	81.9	89.0
DeepSearchQA (accuracy)	83.0	63.7	80.6	60.2	77.1
WideSearch (item-f1)	80.8	-	-	-	72.7
Toolathlon	50.0	54.6	47.2	48.8	27.8
MCPMark	55.9	62.5*	56.7*	55.9*	29.5
Claw Eval (pass^3)	62.3	60.3	70.4	57.8	52.3
Claw Eval (pass@3)	80.9	78.4	82.4	82.9	75.4
APEX-Agents	27.9	33.3	33.0	32.0	11.5
OSWorld-Verified	73.1	75.0	72.7	-	63.3
Coding					
Terminal-Bench 2.0 (Terminus-2)	66.7	65.4*	65.4	68.5	50.8
SWE-Bench Pro	58.6	57.7	53.4	54.2	50.7
SWE-Bench Multilingual	76.7	-	77.8	76.9*	73.0
SWE-Bench Verified	80.2	-	80.8	80.6	76.8
SciCode	52.2	56.6	51.9	58.9	48.7
OJBench (python)	60.6	-	60.3	70.7	54.7
LiveCodeBench (v6)	89.6	-	88.8	91.7	85.0
Reasoning & Knowledge					
HLE-Full	34.7	39.8	40.0	44.4	30.1
AIME 2026	96.4	99.2	96.7	98.3	95.8
HMMT 2026 (Feb)	92.7	97.7	96.2	94.7	87.1
IMO-AnswerBench	86.0	91.4	75.3	91.0*	81.8

Benchmark	Kimi K2.6	GPT-5.4 (xhigh)	Claude Opus 4.6 (max effort)	Gemini 3.1 Pro (thinking high)	Kimi K2.5
GPQA-Diamond	90.5	92.8	91.3	94.3	87.6
Vision					
MMMU-Pro	79.4	81.2	73.9	83.0*	78.5
MMMU-Pro (w/ python)	80.1	82.1	77.3	85.3*	77.7
CharXiv (RQ)	80.4	82.8*	69.1	80.2*	77.5
CharXiv (RQ) (w/ python)	86.7	90.0*	84.7	89.9*	78.7
MathVision	87.4	92.0*	71.2*	89.8*	84.2
MathVision (w/ python)	93.2	96.1*	84.6*	95.7*	85.0
BabyVision	39.8	49.7	14.8	51.6	36.5
BabyVision (w/ python)	68.5	80.2*	38.4*	68.3*	40.5
V* (w/ python)	96.9	98.4*	86.4*	96.9*	86.9

▶ **Footnotes**

4. Native INT4 Quantization

Kimi-K2.6 adopts the same native int4 quantization method as Kimi-K2-Thinking.

5. Deployment

> [!Note] You can access Kimi-K2.6's API on https://platform.moonshot.ai and we provide OpenAI/Anthropic-compatible API for you. To verify the deployment is correct, we also provide the Kimi Vendor Verifier. Currently, Kimi-K2.6 is recommended to run on the following inference engines:

- vLLM
- SGLang
- KTransformers

Kimi-K2.6 has the same architecture as Kimi-K2.5, and the deployment method can be directly reused.

The version requirement for `transformers` is `>=4.57.1, <5.0.0`.

Deployment examples can be found in the Model Deployment Guide.

---

6. Model Usage

The usage demos below demonstrate how to call our official API.

For third-party APIs deployed with vLLM or SGLang, please note that:

> [!Note]
>
> - Chat with video content is an experimental feature and is only supported in our official API for now.
>
> - The recommended `temperature` will be `1.0` for Thinking mode and `0.6` for Instant mode.

- The recommended `top_p` is `0.95`.
- To use instant mode, you need to pass `{'chat_template_kwargs': {"thinking": False}}` in `extra_body`.

Chat Completion

This is a simple chat completion script which shows how to call K2.6 API in Thinking and Instant modes.

```
import openai
import base64
import requests
def simple_chat(client: openai.OpenAI, model_name: str):
    messages = [
        {'role': 'system', 'content': 'You are Kimi, an AI assistant created by Moonshot
        AI.'},
        {
            'role': 'user',
            'content': [
                {'type': 'text', 'text': 'which one is bigger, 9.11 or 9.9? think
        carefully.'}
            ],
        },
    ]
    response = client.chat.completions.create(
        model=model_name, messages=messages, stream=False, max_tokens=4096
    )
    print('====== Below is reasoning content in Thinking Mode ======')
    print(f'reasoning content: {response.choices[0].message.reasoning}')
    print('====== Below is response in Thinking Mode ======')
    print(f'response: {response.choices[0].message.content}')

    # To use instant mode, pass {"thinking" = {"type":"disabled"}}
    response = client.chat.completions.create(
        model=model_name,
        messages=messages,
        stream=False,
        max_tokens=4096,
        extra_body={'thinking': {'type': 'disabled'}},  # this is for official API
        # extra_body= {'chat_template_kwargs': {"thinking": False}}  # this is for
        vLLM/SGLang
    )
    print('====== Below is response in Instant Mode ======')
    print(f'response: {response.choices[0].message.content}')
```

Chat Completion with visual content

K2.6 supports Image and Video input.

The following example demonstrates how to call K2.6 API with image input:

```
import openai
import base64
import requests

def chat_with_image(client: openai.OpenAI, model_name: str):
    url = 'https://huggingface.co/moonshotai/Kimi-K2.6/resolve/main/figures/kimi-logo.png'
    image_base64 = base64.b64encode(requests.get(url).content).decode()
```

```
    messages = [
        {
            'role': 'user',
            'content': [
                {'type': 'text', 'text': 'Describe this image in detail.'},
                {
                    'type': 'image_url',
                    'image_url': {'url': f' {image_base64}'},
                },
            ],
        }
    ]

    response = client.chat.completions.create(
        model=model_name, messages=messages, stream=False, max_tokens=8192
    )
    print('====== Below is reasoning content in Thinking Mode ======')
    print(f'reasoning content: {response.choices[0].message.reasoning}')
    print('====== Below is response in Thinking Mode ======')
    print(f'response: {response.choices[0].message.content}')

    # Also support instant mode if you pass {"thinking" = {"type":"disabled"}}
    response = client.chat.completions.create(
        model=model_name,
        messages=messages,
        stream=False,
        max_tokens=4096,
        extra_body={'thinking': {'type': 'disabled'}},  # this is for official API
        # extra_body= {'chat_template_kwargs': {"thinking": False}}  # this is for
        vLLM/SGLang
    )
    print('====== Below is response in Instant Mode ======')
    print(f'response: {response.choices[0].message.content}')

    return response.choices[0].message.content
```

The following example demonstrates how to call K2.6 API with video input:

```
import openai
import base64
import requests

def chat_with_video(client: openai.OpenAI, model_name:str):
    url = 'https://huggingface.co/moonshotai/Kimi-K2.6/resolve/main/figures/demo_video.mp4'
    video_base64 = base64.b64encode(requests.get(url).content).decode()
    messages = [
        {
            "role": "user",
            "content": [
                {"type": "text","text": "Describe the video in detail."},
                {
                    "type": "video_url",
                    "video_url": {"url": f"data:video/mp4; base64,{video_base64}"},
                },
            ],
        }
    ]

    response = client.chat.completions.create(model=model_name, messages=messages)
    print('====== Below is reasoning content in Thinking Mode ======')
```

```
    print(f'reasoning content: {response.choices[0].message.reasoning}')
    print('====== Below is response in Thinking Mode ======')
    print(f'response: {response.choices[0].message.content}')

    # Also support instant mode if pass {"thinking" = {"type":"disabled"}}
    response = client.chat.completions.create(
        model=model_name,
        messages=messages,
        stream=False,
        max_tokens=4096,
        extra_body={'thinking': {'type': 'disabled'}},  # this is for official API
        # extra_body= {'chat_template_kwargs': {"thinking": False}}  # this is for
        vLLM/SGLang
    )
    print('====== Below is response in Instant Mode ======')
    print(f'response: {response.choices[0].message.content}')
    return response.choices[0].message.content
```

Preserve Thinking

Kimi K2.6 supports `preserve_thinking` mode, which retains full reasoning content across multi-turn interactions and enhances performance in coding agent scenarios.

This feature is disabled by default. The following example demonstrates how to call K2.6 API in `preserve_thinking` mode:

```
def chat_with_preserve_thinking(client: openai.OpenAI, model_name: str):
    messages = [
        {
            "role": "user",
            "content": "Tell me three random numbers."
        },
        {
            "role": "assistant",
            "reasoning_content": "I'll start by listing five numbers: 473, 921, 235, 215,
        222, and I'll tell you the first three.",
            "content": "473, 921, 235"
        },
        {
            "role": "user",
            "content": "What are the other two numbers you have in mind?"
        }
    ]

    response = client.chat.completions.create(
        model=model_name,
        messages=messages,
        stream=False,
        max_tokens=4096,
        extra_body={'thinking': {'type': 'enabled', 'keep': 'all'}},  # this is for official
        API
        # extra_body={"chat_template_kwargs": {"thinking":True, "preserve_thinking": True}},
        # this is for vLLM/SGLang
        # We recommend enabling preserve_thinking only in think mode.
    )
    # the assistant should mention 215 and 222 that appear in the prior reasoning content
    print(f"response: {response.choices[0].message.reasoning}")
    return response.choices[0].message.content
```

Interleaved Thinking and Multi-Step Tool Call

K2.6 shares the same design of Interleaved Thinking and Multi-Step Tool Call as K2 Thinking. For usage example, please refer to the K2 Thinking documentation.

Coding Agent Framework

Kimi K2.6 works best with Kimi Code CLI as its agent framework — give it a try at https://www.kimi.com/code.

---

7. License

Both the code repository and the model weights are released under the Modified MIT License.

---

8. Third Party Notices

See THIRD PARTY NOTICES

---

9. Contact Us

If you have any questions, please reach out at support@moonshot.ai.